

Global Coding Challenge 2026 - Entry Challenge

Kan Chiong Delivery Driver

You are given a city road network with time-dependent traffic obstructions.

Given:

- a start coordinate
- an end coordinate
- a departure time

Compute the **fastest route by travel time** and return:

1. total duration in seconds
2. arrival time
3. ordered list of traversed edge_ids

Challenge Submission

Implement:

def solve(data: str) -> str

- The input data is a JSON string matching the input format below.
- The function must return a JSON string matching the output format.

Input Format

```
{
  "start_coordinate": [x, y],
  "end_coordinate": [x, y],
  "start_time": "ISO-8601",
  "nodes": [[x1, y1], [x2, y2], ...],
  "edges": [
    {
      "edge_id": "string",
      "node1": [x, y],
      "node2": [x, y],
      "base_duration_sec": 0
    }
  ],
  "obstructions": [
    {
      "edge_id": "string",
      "edge": {
        "from": [x, y],
        "to": [x, y]
      },
      "start_time": "ISO-8601",
      "end_time": "ISO-8601",
      "speed_factor": 0.0
    }
  ]
}
```

```
]
}
```

Notes

- Edges are **bidirectional** with the same base duration in both directions.
- Obstructions are **directional** and apply only when both match:
 - edge_id
 - edge.from -> edge.to

Output Format

```
{
  "total_duration_sec": 0,
  "arrival_time": "ISO-8601",
  "path": ["edge_id_1", "edge_id_2", "..."]
}
```

If destination is unreachable:

```
{
  "total_duration_sec": null,
  "arrival_time": null,
  "path": []
}
```

Constraints

- **No waiting** at nodes.
- Coordinates are integer pairs: [x, y].
- base_duration_sec is an integer in [0, 999].
- Cycles are allowed (a node may be revisited).
- If an obstruction becomes active during traversal, only the remaining untraveled portion is affected.
 - Example: 30s edge, obstruction starts 10s after departure, speed_factor=0.5
=> 10 + (20 / 0.5) = 50 seconds total.
- speed_factor = 0.0 means that directed traversal is blocked while active.

Examples

The examples below are illustrative and are not exhaustive.

Example 1

Input

```
{
  "start_coordinate": [0, 0],
  "end_coordinate": [3, 1],
  "start_time": "2026-06-10T08:30:00Z",
  "nodes": [[0, 0], [1, 0], [2, 0], [2, 1], [3, 1]],
  "edges": [
    { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_duration_sec": 60 },
    { "edge_id": "edge_1", "node1": [1, 0], "node2": [2, 0], "base_duration_sec": 60 },
```

```

    { "edge_id": "edge_2", "node1": [2, 0], "node2": [2, 1], "base_duration_sec": 40 },
    { "edge_id": "edge_3", "node1": [2, 1], "node2": [3, 1], "base_duration_sec": 50 },
    { "edge_id": "edge_4", "node1": [1, 0], "node2": [2, 1], "base_duration_sec": 120 }
  ],
  "obstructions": [
    {
      "edge_id": "edge_1",
      "edge": { "from": [1, 0], "to": [2, 0] },
      "start_time": "2026-06-10T08:00:00Z",
      "end_time": "2026-06-10T09:00:00Z",
      "speed_factor": 0.5
    },
    {
      "edge_id": "edge_2",
      "edge": { "from": [2, 1], "to": [2, 0] },
      "start_time": "2026-06-10T08:15:00Z",
      "end_time": "2026-06-10T08:45:00Z",
      "speed_factor": 0.0
    }
  ]
}

```

Output

```

{
  "total_duration_sec": 230,
  "arrival_time": "2026-06-10T08:33:50Z",
  "path": ["edge_0", "edge_4", "edge_3"]
}

```

Explanation

edge_4 is preferred over edge_1 + edge_2 because of active obstruction impact.

- Candidate path ["edge_0", "edge_1", "edge_2", "edge_3"]:
 - edge_0: 60s
 - edge_1: 120s (due to speed_factor=0.5)
 - edge_2: 40s (directional obstruction does not apply in this direction)
 - edge_3: 50s
 - Total: 270s
- Chosen path ["edge_0", "edge_4", "edge_3"]: 60 + 120 + 50 = 230s

Example 2

Input

```

{
  "start_coordinate": [0, 0],
  "end_coordinate": [3, 3],
  "start_time": "2026-06-10T08:30:00Z",
  "nodes": [[0, 0], [1, 0], [2, 0], [2, 1], [3, 1]],

```

```

"edges": [
  { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_duration_sec": 60 },
  { "edge_id": "edge_1", "node1": [1, 0], "node2": [2, 0], "base_duration_sec": 60 },
  { "edge_id": "edge_2", "node1": [2, 0], "node2": [2, 1], "base_duration_sec": 40 },
  { "edge_id": "edge_3", "node1": [2, 1], "node2": [3, 1], "base_duration_sec": 50 },
  { "edge_id": "edge_4", "node1": [1, 0], "node2": [2, 1], "base_duration_sec": 120 }
],
"obstructions": [
  {
    "edge_id": "edge_1",
    "edge": { "from": [1, 0], "to": [2, 0] },
    "start_time": "2026-06-10T08:00:00Z",
    "end_time": "2026-06-10T09:00:00Z",
    "speed_factor": 0.5
  },
  {
    "edge_id": "edge_2",
    "edge": { "from": [2, 1], "to": [2, 0] },
    "start_time": "2026-06-10T08:15:00Z",
    "end_time": "2026-06-10T08:45:00Z",
    "speed_factor": 0.0
  }
]
}

```

Output

```

{
  "total_duration_sec": null,
  "arrival_time": null,
  "path": []
}

```

Explanation

end_coordinate is unreachable, so the expected result is the null-duration no-path response.

Example 3 (No Waiting + Cycling)

Input

```

{
  "start_coordinate": [0, 0],
  "end_coordinate": [2, 0],
  "start_time": "2026-06-10T08:30:00Z",
  "nodes": [[0, 0], [1, 0], [2, 0]],
  "edges": [
    { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_duration_sec": 10 },
    { "edge_id": "edge_1", "node1": [1, 0], "node2": [2, 0], "base_duration_sec": 10 },

```

```
{ "edge_id": "edge_2", "node1": [0, 0], "node2": [2, 0], "base_duration_sec": 20 }
],
"obstructions": [
{
  "edge_id": "edge_1",
  "edge": { "from": [1, 0], "to": [2, 0] },
  "start_time": "2026-06-10T08:30:10Z",
  "end_time": "2026-06-10T08:30:20Z",
  "speed_factor": 0.0
},
{
  "edge_id": "edge_1",
  "edge": { "from": [1, 0], "to": [2, 0] },
  "start_time": "2026-06-10T08:30:30Z",
  "end_time": "2026-06-10T08:30:40Z",
  "speed_factor": 0.0
},
{
  "edge_id": "edge_2",
  "edge": { "from": [0, 0], "to": [2, 0] },
  "start_time": "2026-06-10T08:30:00Z",
  "end_time": "2026-06-10T08:32:00Z",
  "speed_factor": 0.2
}
]
}
```

Output

```
{
  "total_duration_sec": 60,
  "arrival_time": "2026-06-10T08:31:00Z",
  "path": ["edge_0", "edge_0", "edge_0", "edge_0", "edge_0", "edge_1"]
}
```

Explanation

No waiting is allowed, so the route repeatedly traverses edge_0 to advance time until edge_1 is traversable.

- When reaching [1, 0] at 08:30:10 or 08:30:30, traversing edge_1 toward [2, 0] is blocked.
- Alternative edge_2 during its obstruction window is slower ($20 / 0.2 = 100\text{s}$)
- Best no-wait option is to cycle using edge_0 and then take edge_1

Example 4 (No Waiting + Blocked at Start)

Input

```
{
  "start_coordinate": [0, 0],
```

```
"end_coordinate": [1, 0],
"start_time": "2026-06-10T08:30:00Z",
"nodes": [[0, 0], [1, 0]],
"edges": [
  { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_duration_sec": 60 }
],
"obstructions": [
  {
    "edge_id": "edge_0",
    "edge": { "from": [0, 0], "to": [1, 0] },
    "start_time": "2026-06-10T08:00:00Z",
    "end_time": "2026-06-10T09:00:00Z",
    "speed_factor": 0.0
  }
]
```

Output

```
{
  "total_duration_sec": null,
  "arrival_time": null,
  "path": []
}
```

Explanation

Waiting is not allowed, and the only outgoing move from start_coordinate is blocked (speed_factor=0.0) at departure time, so no valid route exists.